

(Applications Development and Emerging Technologies)

PRE-SUMMATIVE ASSESSMENT

4

PHP PRE DEFINED FUNCTIONS

Student Name / Group Name:	Kirk Marcus Garing	
Members (if Group):	Name	Role
Section:	TC27	
Professor:	Sir Calleja	

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements.

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Understand and apply best practices and standards in the development of website.

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- To know how to include separate PHP code in the main page for code enhancement.
- To be familiar with the use of common predefined function such as define, include, and require.
- To use different available mathematical function for manipulating numbers.

IV. BACKGROUND INFORMATION

PHP Predefined Functions

Using Constant

`define()` functions

used to declare constants. A constant can only be assigned a scalar value, like a string or a number. A constant's value cannot be changed.

Syntax:

```
define('NAME', 'value');
```

Example:

```
<?php
define('MAX_VALUE',10);
for($i=5;$i<MAX_VALUE;$i++){
    echo $i." ";
}
//MAX_VALUE = 20; is invalids
?>
```

Output:

5 6 7 8 9

PHP Constants

Constants are like variables except that once they are defined they cannot be changed or undefined.

A constant is an identifier (name) for a simple value. The value cannot be changed during the script.

A valid constant name starts with a letter or underscore (no \$ sign before the constant name).

Note: Unlike variables, constants are automatically global across the entire script.

Create a PHP Constant

To create a constant, use the `define()` function.

Syntax

```
define(name, value, case-insensitive)
```

Parameters:

- *name*: Specifies the name of the constant
- *value*: Specifies the value of the constant
- *case-insensitive*: Specifies whether the constant name should be case-insensitive. Default is false

Example

Create a constant with a **case-sensitive** name:

```
<?php  
define("GREETING", "Welcome to PHP Programming!");  
echo GREETING;  
?>
```

PHP Include Files

The `include` (or `require`) statement takes all the text/code/markup that exists in the specified file and copies it into the file that uses the include statement.

Including files is very useful when you want to include the same PHP, HTML, or text on multiple pages of a website.

PHP include and require Statements

It is possible to insert the content of one PHP file into another PHP file (before the server executes it), with the include or require statement.

The include and require statements are identical, except upon failure:

- `require` will produce a fatal error (E_COMPILE_ERROR) and stop the script
- `include` will only produce a warning (E_WARNING) and the script will continue

So, if you want the execution to go on and show users the output, even if the include file is missing, use the include statement. Otherwise, in case of Framework, CMS, or a complex PHP application coding, always use the require statement to include a key file to the flow of execution. This will help avoid compromising your application's security and integrity, just in-case one key file is accidentally missing.

Including files saves a lot of work. This means that you can create a standard header, footer, or menu file for all your web pages. Then, when the header needs to be updated, you can only update the header include file.

Syntax

```
include 'filename';
```

or

```
require 'filename';
```

PHP include Examples

Example 1

Assume we have a standard footer file called "footer.php", that looks like this:

```
<?php
echo "<p>Copyright &copy; 1999-" . date("Y") . " W3Schools.com</p>";
?>
```

To include the footer file in a page, use the `include` statement:

Example

```
<html>
<body>

<h1>Welcome to my home page!</h1>
<p>Some text.</p>
<p>Some more text.</p>
<?php include 'footer.php';?>

</body>
</html>
```

Example 2

Assume we have a standard menu file called "menu.php":

```
<?php
echo '<a href="/default.asp">Home</a> -
<a href="/html/default.asp">HTML Tutorial</a> -
<a href="/css/default.asp">CSS Tutorial</a> -
<a href="/js/default.asp">JavaScript Tutorial</a> -
<a href="default.asp">PHP Tutorial</a>';
?>
```

All pages in the Web site should use this menu file. Here is how it can be done (we are using a <div> element so that the menu easily can be styled with CSS later):

Example

```
<html>
<body>

<div class="menu">
<?php include 'menu.php';?>
</div>

<h1>Welcome to my home page!</h1>
<p>Some text.</p>
<p>Some more text.</p>
```

```
</body>
</html>
```

Example 3

Assume we have a file called "vars.php", with some variables defined:

```
<?php
$color='red';
$car='BMW';
?>
```

Then, if we include the "vars.php" file, the variables can be used in the calling file:

Example

```
<html>
<body>

<h1>Welcome to my home page!</h1>
<?php include 'vars.php';
echo "I have a $color $car.";
?>

</body>
</html>
```

PHP include vs. require

The `require` statement is also used to include a file into the PHP code.

However, there is one big difference between include and require; when a file is included with the `include` statement and PHP cannot find it, the script will continue to execute:

Example

```
<html>
<body>

<h1>Welcome to my home page!</h1>
```

```
<?php include 'noFileExists.php';  
echo "I have a $color $car."  
?>
```

```
</body>  
</html>
```

If we do the same example using the `require` statement, the `echo` statement will not be executed because the script execution dies after the `require` statement returned a fatal error:

Example

```
<html>  
<body>  
  
<h1>Welcome to my home page!</h1>  
<?php require 'noFileExists.php';  
echo "I have a $color $car."  
?>
```

```
</body>  
</html>
```

PHP String Functions

In this chapter we will look at some commonly used functions to manipulate strings.

strlen() - Return the Length of a String

The PHP `strlen()` function returns the length of a string.

Example

Return the length of the string "Hello world!":

```
<?php  
echo strlen("Hello world!"); // outputs 12  
?>
```

str_word_count() - Count Words in a String

The PHP `str_word_count()` function counts the number of words in a string.

Example

Count the number of word in the string "Hello world!":

```
<?php
echo str_word_count("Hello world!"); // outputs 2
?>
```

strrev() - Reverse a String

The PHP `strrev()` function reverses a string.

Example

Reverse the string "Hello world!":

```
<?php
echo strrev("Hello world!"); // outputs !dlrow olleH
?>
```

strpos() - Search For a Text Within a String

The PHP `strpos()` function searches for a specific text within a string. If a match is found, the function returns the character position of the first match. If no match is found, it will return FALSE.

Example

Search for the text "world" in the string "Hello world!":

```
<?php
echo strpos("Hello world!", "world"); // outputs 6
?>
```


Tip: The first character position in a string is 0 (not 1).

str_replace() - Replace Text Within a String

The PHP `str_replace()` function replaces some characters with some other characters in a string.

Example

Replace the text "world" with "Dolly":

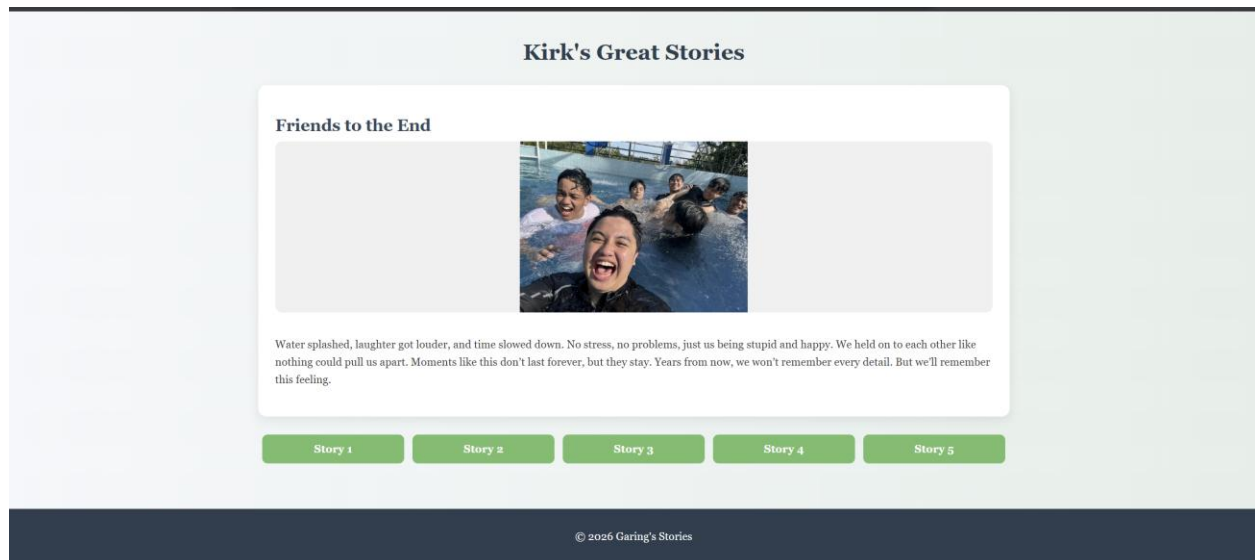
```
<?php
echo str_replace("world", "Dolly", "Hello world!"); // outputs Hello
Dolly!
?>
```

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

1. Create 5 different short story webpages and Convert your stories into a web based form, use `include()` and `require()` functions to connect all pages integrate with HTML and CSS (Put images on the stories). See the format below:

Story 1	Story 2	Story 3	Story 4	Story 5
---------	---------	---------	---------	---------



2. Creation of String Function in PHP, create an array with 20 different names and get the following:
 - Number of characters of each name include spaces
 - Change the first character of each name into upper case
 - Replace all vowels with @ symbol
 - Check the position of letter a
 - Change the name into reverse formate
- Please see the sample below

List of names					
Name	Number of characters	Uppercase first character	Replace vowels with @	Check position of character "a"	Reverse name
chrisa	6	Chrisa	chr@s@	5	asirhc

Snip and paste your source codes here. Snip it directly from the IDE so that colors of the codes are preserved for readability. Include additional pages if necessary.

List of names					
Name	Number of characters	Uppercase first character	Replace vowels with @	Check position of character "a"	Reverse name
chrisa	6	Chrisa	chr@s@a	6	asirhc
john doe	8	John doe	j@hnd@e	Not found	eod nhoj
mary ann	8	Mary ann	m@ry @nn	2	nna yram
alexander	9	Alexander	@!@x@nd@r	1	rednaxela
elizabeth	9	Elizabeth	@!@z@b@th	5	htebazile
michael	7	Michael	m@ch@e@l	5	leahcim
sarah connor	12	Sarah connor	s@r@h c@nn@r	2	ronnoc haras
david	5	David	d@v@d	2	divad
emily	5	Emily	@m@ily	Not found	ylime
james bond	10	James bond	j@m@s b@nd	2	dnob semaj
jessica	7	Jessica	j@ss@c@e	7	ecissej
brian	5	Brian	br@ @n	4	naibr
amanda	6	Amanda	@m@nd@	1	adnama
kevin	5	Kevin	k@v@n	Not found	nivek
rachel	6	Rachel	r@ch@l	2	lehcar
oliver	6	Oliver	@!@v@r	Not found	revilo
sophia	6	Sophia	s@ph@e@	6	aihpos
william	7	William	w@ll@ @m	6	mailliw
natalie	7	Natalie	n@t@!@e@	2	eiltan
ethan	5	Ethan	@th@n	4	nahte

VII. QUESTION AND ANSWER

1. What is Constant in PHP

When I'm coding in PHP, I look at a constant as basically a locked variable that I can set once at the start, and it stays exactly the same no matter what happens later in my script. Unlike the regular variables I use that start with a dollar sign and can be overwritten anytime, constants don't use a dollar sign at all, and I'll actually get an error if I try to change their value or undefine them later on. In my assignments, I usually create them using either the define function, which works pretty much anywhere in my code, or the const keyword, which looks a lot cleaner but is usually restricted to the top-level scope or inside my object-oriented classes. I find them super useful for storing global configuration settings that I don't want to accidentally mess up later, like database credentials, API keys, or mathematical values like Pi. Another really cool thing I noticed is that they are automatically global, meaning I can access them inside any of my functions or classes without having to pass them through arguments or use the global keyword. PHP even gives me these built-in "magic constants" like double underscore LINE double underscore that automatically change based on where I write them, which saves me a ton of time when I'm debugging and need to track down exactly where my script is running.

2. What is the difference between include and require? Explain

When I'm working on PHP projects, the main difference I notice between include and require is how they handle errors when a file is missing. If I use include and PHP can't find the file I'm asking for, it will just throw a warning but let the rest of my script keep running. This is super helpful for things like a footer or a sidebar where the website can still function even if that specific part fails to load. On the other hand, if I use require and the file is missing, PHP treats it as a fatal error and completely

stops running the script right then and there. I usually save require for absolute essentials that the rest of my code depends on to work, like a database connection file or a core configuration script, because there is no point in running the rest of the page if those aren't loaded. On top of that, I also use include_once and require_once in my assignments when I want to make sure PHP only loads a file one time, which prevents me from accidentally redeclaring functions or causing weird loops if I accidentally call the same file twice.

3. What are the different string functions? Explain each

When I'm working on PHP assignments, I use a bunch of different built-in string functions to manipulate and format text depending on what my code needs to do. One of the most basic ones I use all the time is strlen(), which just counts the total number of characters in a string, including spaces and punctuation, which is super useful for validating form inputs like passwords. If I need to change the casing of my text, I use functions like strtolower() to make everything lowercase, strtoupper() to make it all uppercase, or ucfirst() when I just want to capitalize the very first letter of the sentence. When I need to clean up user input, like stripping out accidental spaces someone typed at the beginning or end of a username, trim() is my go-to function. For searching and modifying text, I rely on strpos() to find the specific starting position of a word or character inside a longer string, and if I need to swap a specific word out for another one, I use str_replace(). Finally, whenever a project requires me to flip a piece of text completely backwards—like checking for palindromes or doing basic text reversing exercises—I just throw the string into strrev() and it flips it instantly.

VIII. REFERENCES

1. <https://www.w3schools.com/css/>
2. <https://www.w3schools.com/html/>
3. https://www.w3schools.com/php/php_variables.asp
4. https://www.w3schools.com/php/php_constants.asp
5. https://www.w3schools.com/php/php_includes.asp
6. https://www.w3schools.com/php/php_string.asp

Note: The following rubrics/metrics will be used to grade students' output.

Program (100 pts.)	(Excellent)	(Good)	(Fair)	(Poor)
Program execution (20pts)	Program executes correctly with no syntax or runtime errors (18-20pts)	Program executes with less than 3 errors (15-17pts)	Program executes with more than 3 errors (12-14pts)	Program does not execute (10-11pts)
Correct output (20pts)	Program displays correct output with no errors (18-20pts)	Output has minor errors (15-17pts)	Output has multiple errors (12-14pts)	Output is incorrect (10-11pts)

Design of output (10pts)	Program displays more than expected (10pts)	Program displays minimally expected output (8-9pts)	Program does not display the required output (6-7pts)	Output is poorly designed (5pts)
Design of logic (20pts)	Program is logically well designed (18-20pts)	Program has slight logic errors that do no significantly affect the results (15-17pts)	Program has significant logic errors (3-5pts)	Program is incorrect (10-11pts)
Standards (20pts)	Program code is stylistically well designed (18-20pts)	Few inappropriate design choices (i.e. poor variable names, improper indentation) (15-17pts)	Several inappropriate design choices (i.e. poor variable names, improper indentation) (12-14pts)	Program is poorly written (10-11pts)
Delivery (10pts)	The program was delivered on time. (10pts)	The program was delivered a day after the deadline. (8-9pts)	The program was delivered two days after the deadline. (6-7pts)	The program was delivered more than two days after the deadline. (5pts)